SQL Advanced Practical

by Ariane Heinz

A - KeepCoding Database Implementation....2

Challenge 1: E-R Diagram for KeepCoding.... 2

Challenge 2: KeepCoding Database implementation in PostGreSQL.... 4

B - Customer Service IVR Challenges....6

Challenge 3: Create table ivr_detail.... 6

Challenge 4: Generate field vdn_aggregation....7

Challenge 5: Generate document_type & document_identification.... 8

Challenge 6: Generate customer_phone.... 9

Challenge 7: Generate billing_account_id....10

Challenge 8: Generate masiva_lg.... 11

Challenge 9: Generate info_by_phone_lg....12

Challenge 10: Generate info_by_dni_lg.... 13

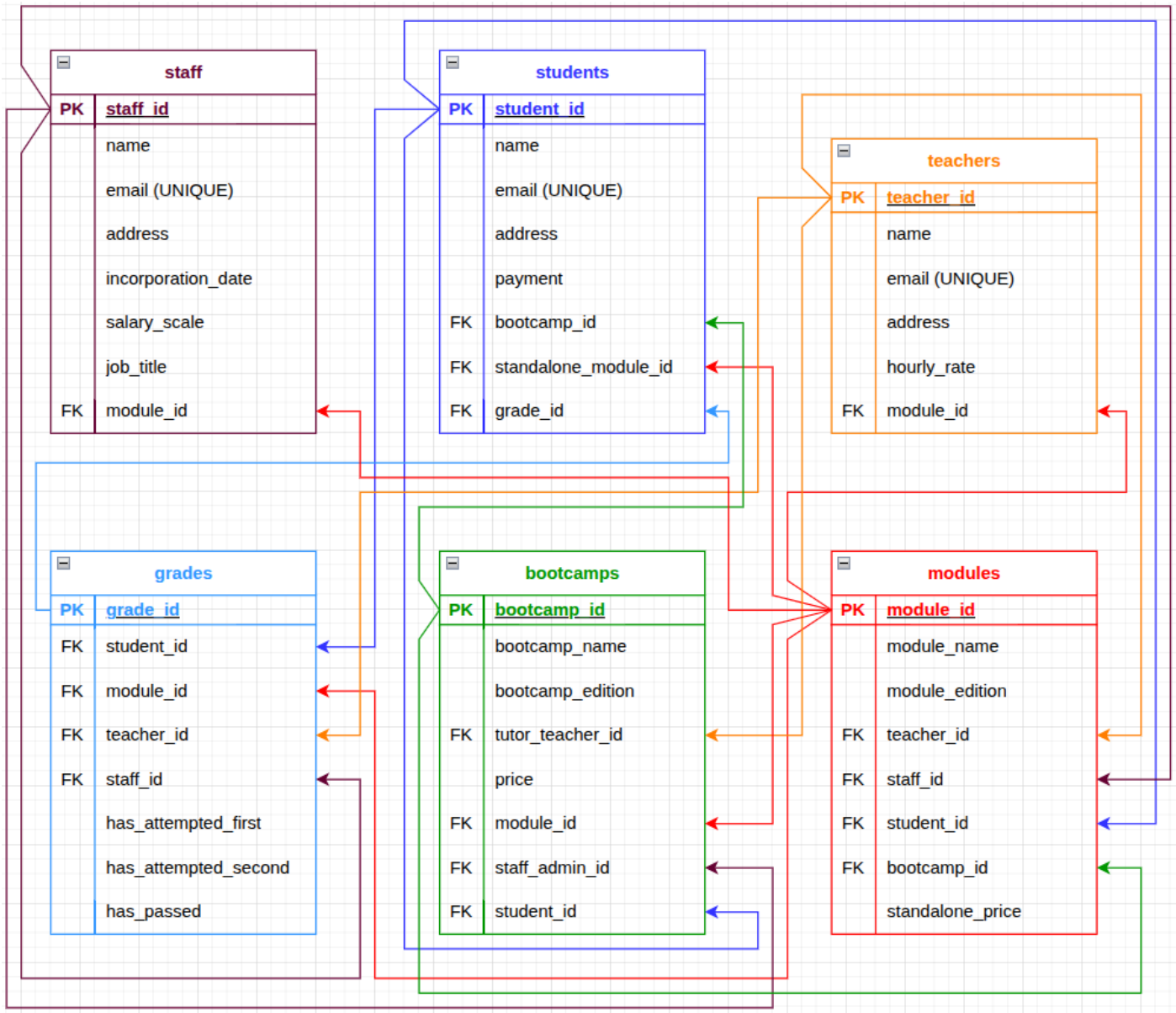
Challenge 11: Generate repeated_phone_24H & cause_recall_phone_24H.... 14

Challenge 12: Create table ivr_summary.... 15

Challenge 13: Create function clean_integer.... 17

A - KeepCoding Database Implementation

Challenge 1: E-R Diagram for KeepCoding



<u>Explanation</u>:
I have subdivided KeepCoding into six tables: three related to people and three to coursework. Each table is displayed in a different color, to facilitate legibility.

People:

- ***Staff*** refers to people who are on a salary, meaning they are working full-time for KeepCoding. It could be systems maintenance, HR, admin or even teachers who are employed as their only job. Each person is on a specific salary scale based on their job title, qualifications and contractual agreement, and I have also taken into account their incorporation year which may impact their benefits. The PK connects with *bootcamps*, *modules* and *grades*.
- ***Teachers*** refers to those people who are teaching specific modules and who are paid by the hour of instruction at a specific hourly rate. Similarly, the PK feeds *bootcamps*, *modules* and *grades* and the table is fed the module_ids that the teacher is associated with. It could be argued that this table could be combined with the *staff* table since its structure is very similar. However, I feel that for questions of

access to sensitive information and data protection, staff may need privileges that should not be awarded to external teachers.

- ***Students*** obviously refers to the students taking the courses, and I placed the students at the center since without students KeepCoding ceases to exist. Again, the PK feeds into bootcamps, modules, and grades. I have planned for the possibility that students may take stand-alone modules, either in addition to a bootcamp or without being signed up to a full bootcamp. The table is fed as FK a bootcamp_id, a grade_id and a stand_alone_module_id. I also included a field for "payment" which has to do with how much the student has paid so far, so KeepCoding can keep track of what is still owed. I'm not sure how this would take place in real life, as I don't have any experience with this.

Coursework:

- ***Modules*** is probably the most important table, as it seems to me that KeepCoding revolves around a series of modules. Thus the PK feeds into all other tables: the *teachers*, *students* and *staff* that are associated with the module, and also the *bootcamps* that use that module and the *grades*. The *modules* table is fed primary keys from all other tables except for *grades*, which only go to *students*. Modules can be taken by students as a stand_alone course or as part of a bootcamp.
- **Bootcamps** refers to a set of modules that combine to make a certification into a domain of data science. They are associated with *students*, *staff*, tutor *teachers* and *modules* and therefore receive as foreign keys the ids associated with those. The PK feeds into *students* and *modules* exclusively.
- ***Grades*** is a fairly separate and stand-alone table that receives as foreign keys the student_id, module_id, teacher_id and staff_id (mostly for cases in which the teacher is a member of staff, or in case a member of staff has to fix an error). Each grade_id (PK) only feeds into the *students* table, so the information about grades can be easily associated with each student.

Other comments:

- I could have added another table for *interested* people or *alumni*, that is people who might not yet or no longer be in the active database of KeepCoding but who may receive promotions or marketing information of some sort. However, I decided to keep it simple.
- In terms of the level of detail, I believe an actual database would have many more fields, such as separating "name" into first, middle and last name, or into first name (*nombre*) and two last names (*1r apellido, 2º apellido*), or such as including the field "phone" as a VARCHAR(20). I tried to keep it simple so the diagram does not become too difficult to read. Also, adding the extra fields would just make the exercise longer, but not teach anything meaningful, in my humble opinion.
- In terms of linking the *students* table and the *grades* table, I remember a class conversation on which way the arrow should go. I decided that student_id from the *students* table will go to *grades* but the grade_id from *grades* will go to *students*. This makes sense to me because the *grades* table is the central place where all grades are stored as a PK, and then each grade is assigned to the right student as a FK, for each module the student has coursed.
- For all people-related people, I have made the "email" field unique to ensure that there are no same emails that refer to different people, since it would be dangerous in terms of data protection to send personal information to the wrong person.

Challenge 2: KeepCoding Database implementation in PostGreSQL

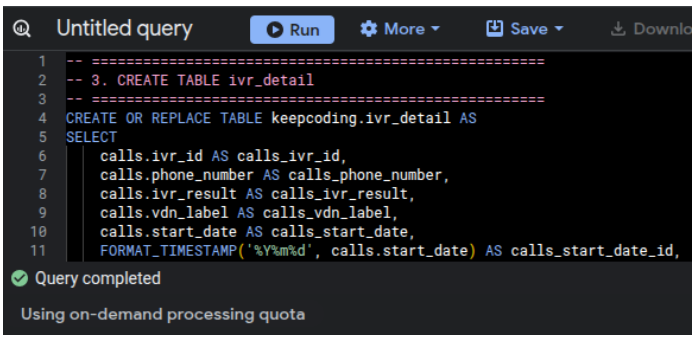
```
CREATE TABLE staff (
  staff_id SERIAL PRIMARY KEY,
  name VARCHAR(255),
  email VARCHAR(255) UNIQUE NOT NULL,
  address VARCHAR(255),
  incorporation_date DATE,
  salary_scale INT,
  job_title VARCHAR(255),
  module_id INT,
  FOREIGN KEY (module_id) REFERENCES modules(module_id)
);

CREATE TABLE students (
  student_id SERIAL PRIMARY KEY,
  name VARCHAR(255),
  email VARCHAR(255) UNIQUE NOT NULL,
  address VARCHAR(255),
  payment DECIMAL(10, 2),  -- Increased precision for monetary values
  bootcamp_id INT,
  standalone_module_id INT,
  grade_id INT,
  FOREIGN KEY (bootcamp_id) REFERENCES bootcamps(bootcamp_id),
  FOREIGN KEY (standalone_module_id) REFERENCES modules(module_id),
  FOREIGN KEY (grade_id) REFERENCES grades(grade_id)
);

CREATE TABLE teachers (
  teacher_id SERIAL PRIMARY KEY,
  name VARCHAR(255),
  email VARCHAR(255) UNIQUE NOT NULL,
  address VARCHAR(255),
  hourly_rate DECIMAL(10, 2),
  module_id INT,
  FOREIGN KEY (module_id) REFERENCES modules(module_id)
);

CREATE TABLE grades (
  grade_id SERIAL PRIMARY KEY,
  student_id INT,
  module_id INT,
  teacher_id INT,
  staff_id INT,
  has_attempted_first BOOLEAN,
  has_attempted_second BOOLEAN,
  has_passed BOOLEAN,
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (module_id) REFERENCES modules(module_id),
  FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id),
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id)
```

```
);

CREATE TABLE bootcamps (
  bootcamp_id SERIAL PRIMARY KEY,
  bootcamp_name VARCHAR(255),
  bootcamp_edition VARCHAR(10),
  tutor_teacher_id INT,
  price DECIMAL(10, 2),
  module_id INT,
  staff_admin_id INT,
  student_id INT,
  FOREIGN KEY (tutor_teacher_id) REFERENCES teachers(teacher_id),
  FOREIGN KEY (module_id) REFERENCES modules(module_id),
  FOREIGN KEY (staff_admin_id) REFERENCES staff(staff_id),
  FOREIGN KEY (student_id) REFERENCES students(student_id)
);

CREATE TABLE modules (
  module_id SERIAL PRIMARY KEY,
  module_name VARCHAR(255),
  module_edition VARCHAR(10),
  teacher_id INT,
  staff_id INT,
  student_id INT,
  bootcamp_id INT,
  standalone_price DECIMAL(10, 2),
  FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id),
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id),
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (bootcamp_id) REFERENCES bootcamps(bootcamp_id)
);
```

B - Customer Service IVR Challenges

Challenge 3: Create table ivr_detail

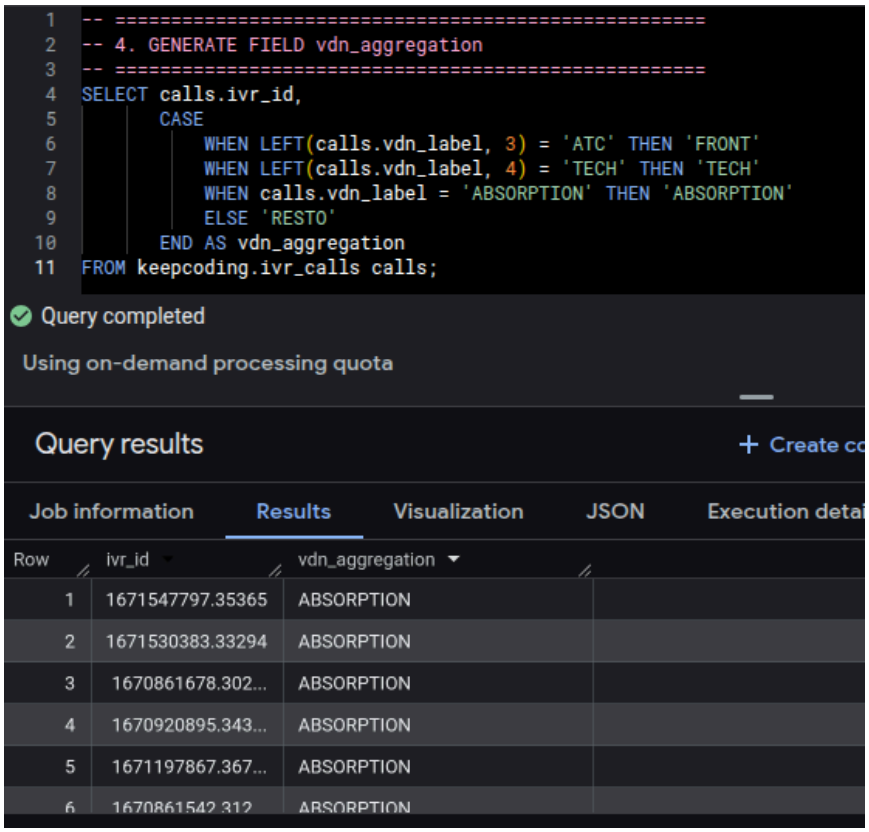
For this challenge, I use LEFT JOINS ON ivr_id and module_sequece to combine the tables ivr_calls, ivr_modules, and ivr_steps.
I am formatting the start and end dates as yyyymmdd. Additionally, I am renaming the fields coming from the ivr_calls table with “calls_” prefixes, so they match the instructions exactly.




```
-- ==========
-- 3. CREATE TABLE ivr_detail
-- ==========
CREATE OR REPLACE TABLE keepcoding.ivr_detail AS
SELECT
    calls.ivr_id AS calls_ivr_id,
    calls.phone_number AS calls_phone_number,
    calls.ivr_result AS calls_ivr_result,
    calls.vdn_label AS calls_vdn_label,
    calls.start_date AS calls_start_date,
    FORMAT_TIMESTAMP('%Y%m%d', calls.start_date) AS calls_start_date_id,
    calls.end_date AS calls_end_date,
    FORMAT_TIMESTAMP('%Y%m%d', calls.end_date) AS calls_end_date_id,
    calls.total_duration AS calls_total_duration,
    calls.customer_segment AS calls_customer_segment,
    calls.ivr_language AS calls_ivr_language,
    calls.steps_module AS calls_steps_module,
    calls.module_aggregation AS calls_module_aggregation,
    modules.module_sequece,
    modules.module_name,
    modules.module_duration,
    modules.module_result,
    steps.step_sequence,
    steps.step_name,
    steps.step_result,
    steps.step_description_error,
    steps.document_type,
    steps.document_identification,
    steps.customer_phone,
    steps.billing_account_id
FROM keepcoding.ivr_calls calls
LEFT JOIN keepcoding.ivr_modules modules
    ON calls.ivr_id = modules.ivr_id
LEFT JOIN keepcoding.ivr_steps steps
    ON modules.ivr_id = steps.ivr_id
    AND modules.module_sequece = steps.module_sequece;
```

Challenge 4: Generate field vdn_aggregation

To produce a general category of VDN labels per call based on their prefix, I am using a CASE clause: if the first 3 characters of the left side of the vdn_label field coincide with ‘ATC’, then the vdn_aggregation field will have the value ‘FRONT’. And similarly with the other cases.

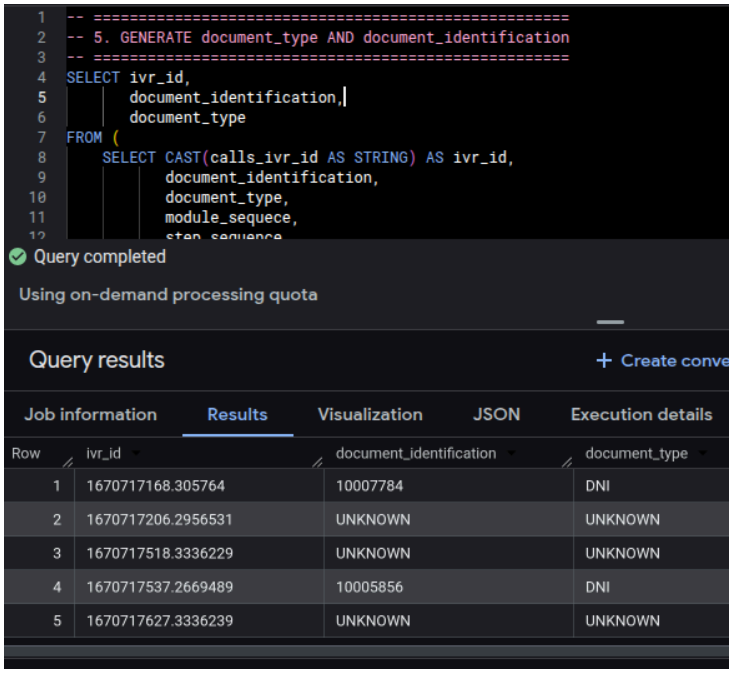



```
-- ==========
-- 4. GENERATE FIELD vdn_aggregation
-- ==========
SELECT calls.ivr_id,
    CASE
        WHEN LEFT(calls.vdn_label, 3) = 'ATC' THEN 'FRONT'
        WHEN LEFT(calls.vdn_label, 4) = 'TECH' THEN 'TECH'
        WHEN calls.vdn_label = 'ABSORPTION' THEN 'ABSORPTION'
        ELSE 'RESTO'
    END AS vdn_aggregation
FROM keepcoding.ivr_calls calls;
```

Challenge 5: Generate document_type & document_identification

Here I extract only one document type and one ID per call, selecting the most relevant information.

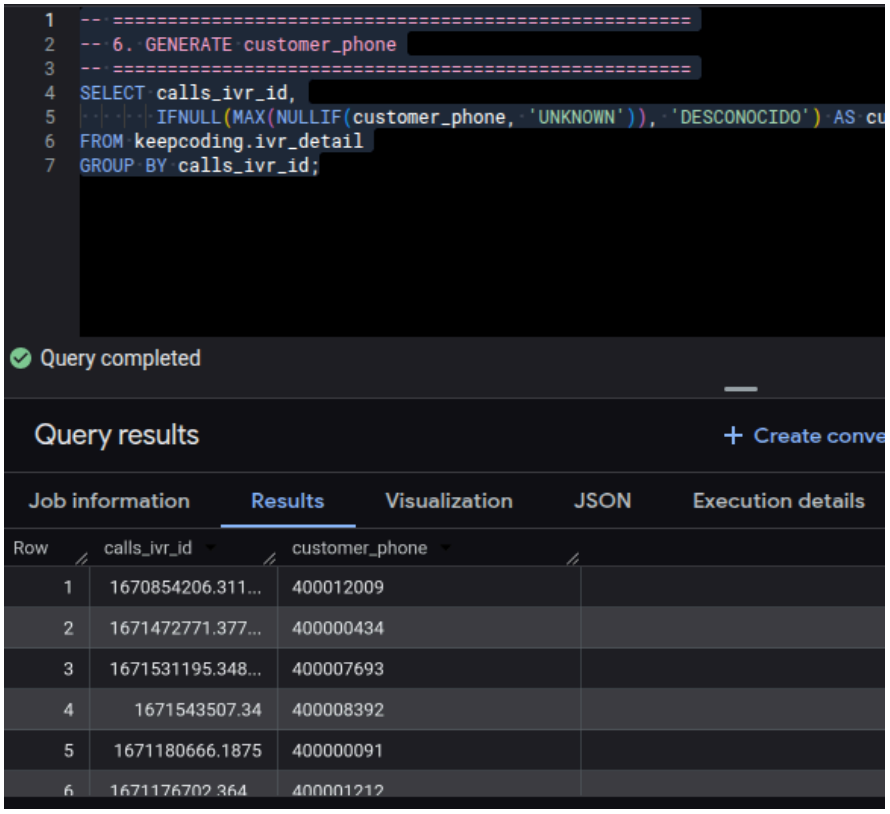
- The inner SELECT statement retrieves data from the ivr_detail table.
- The IF statement checks the value of document_identification: if it is **not** ‘UNKNOWN’ or ‘DESCONOCIDO’, it assigns 1 to has_document_lg (indicating that there is a valid document identification); otherwise, it assigns 0.
- The QUALIFY command is used to filter. ROW_NUMBER() assigns consecutive row numbers within a PARTITION of ivr_id (cast as ‘string’ so BigQuery can perform the partitions).
- The ORDER BY clause effectively selects the best suited row for this ivr_id: it prioritizes rows where has_document_lg = 1, if prioritizes the one(s) with the largest module_sequece, and next it prioritizes the highest step_sequence.
- The output is a table with three columns: ivr_id, document_identification, and document_type.




```
-- ==========
-- 5. GENERATE document_type AND document_identification
-- ==========
SELECT ivr_id,
document_identification,
document_type
FROM (
SELECT CAST(calls_ivr_id AS STRING) AS ivr_id,
document_identification,
document_type,
module_sequece,
step_sequence,
IF(document_identification NOT IN ('UNKNOWN', 'DESCONOCIDO'), 1, 0) AS has_document_lg
FROM keepcoding.ivr_detail
QUALIFY ROW_NUMBER() OVER(
PARTITION BY CAST(calls_ivr_id AS STRING)
ORDER BY has_document_lg DESC, module_sequece DESC, step_sequence DESC
) = 1
);
```

Challenge 6: Generate customer_phone

The goal is to identify a customer's phone for each call. This solution selects and groups by ivr_id, then the different phone values associated with that ivr_id are evaluated: each 'UNKNOWN' becomes 'NULL', and the maximum value of the valid phone numbers is selected as customer_phone. If no valid phone numbers are found, the customer_phone defaults to 'DESCONOCIDO'.




```
-- ==========
-- 6. GENERATE customer_phone
-- ==========
SELECT calls_ivr_id,
    IFNULL(MAX(NULLIF(customer_phone, 'UNKNOWN')), 'DESCONOCIDO') AS customer_phone
FROM keepcoding.ivr_detail
GROUP BY calls_ivr_id;
```

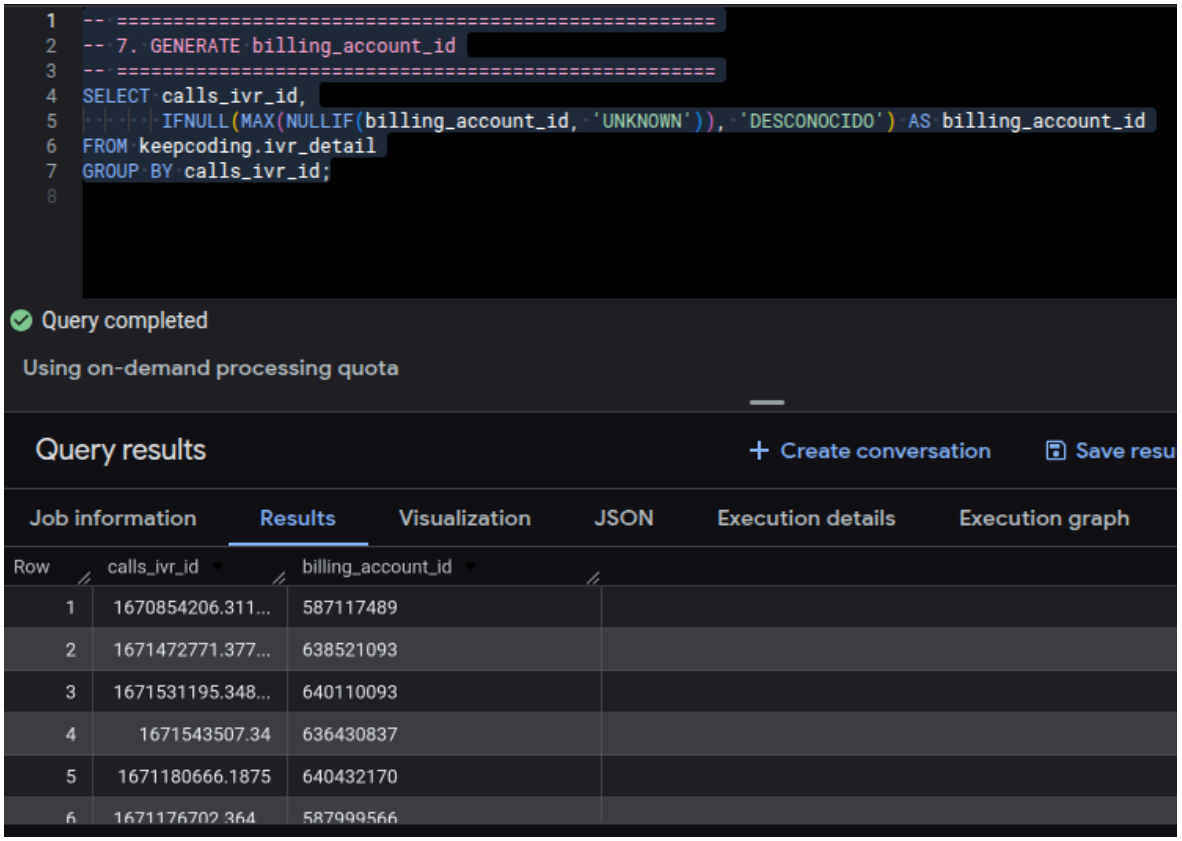
Challenge 7: Generate billing_account_id

The goal is to retrieve a single billing account ID per call.

The inner NULLIF() checks if billing_account_id is ‘UNKNOWN’. If so, it returns NULL; otherwise it returns the value of billing_account_id. Then MAX() finds the maximum value of billing_account_id (after excluding all unknowns which are now NULL).

The outer IFNULL() handles cases where there is no valid billing account id.

GROUP BY calls_ivr_id enforces that, for each unique calls_ivr_id, the query will compute only one billing_account_id (the highest value one as explained above with MAX()).

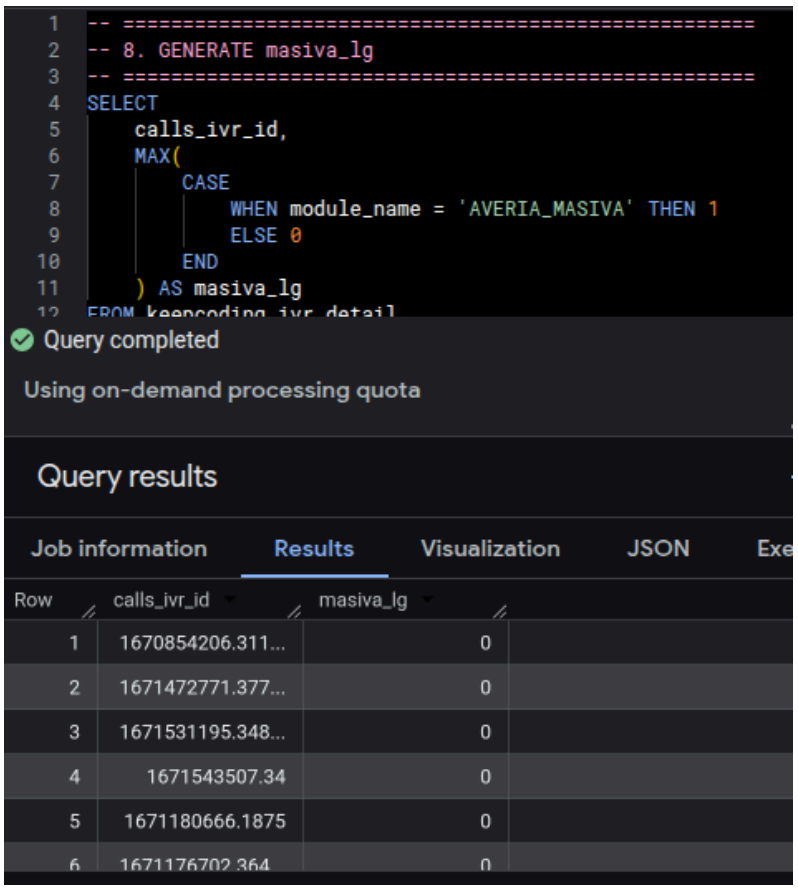



```
-- ==========
-- 7. GENERATE billing_account_id
-- ==========
SELECT calls_ivr_id,
    IFNULL(MAX(NULLIF(billing_account_id, 'UNKNOWN')), 'DESCONOCIDO') AS billing_account_id
FROM keepcoding.ivr_detail
GROUP BY calls_ivr_id;
```

Challenge 8: Generate masiva_lg

To solve this challenge, I am using CASE to create the flag masiva_lg that indicates whether a call passed through the module ‘AVERIA_MASIVA’ or not.

The query groups the data by ivr_id and checks the name of each module. I am using MAX(IF(...)) so that if the module appears at least once in the call, the flag becomes 1, and otherwise it continues to be 0.

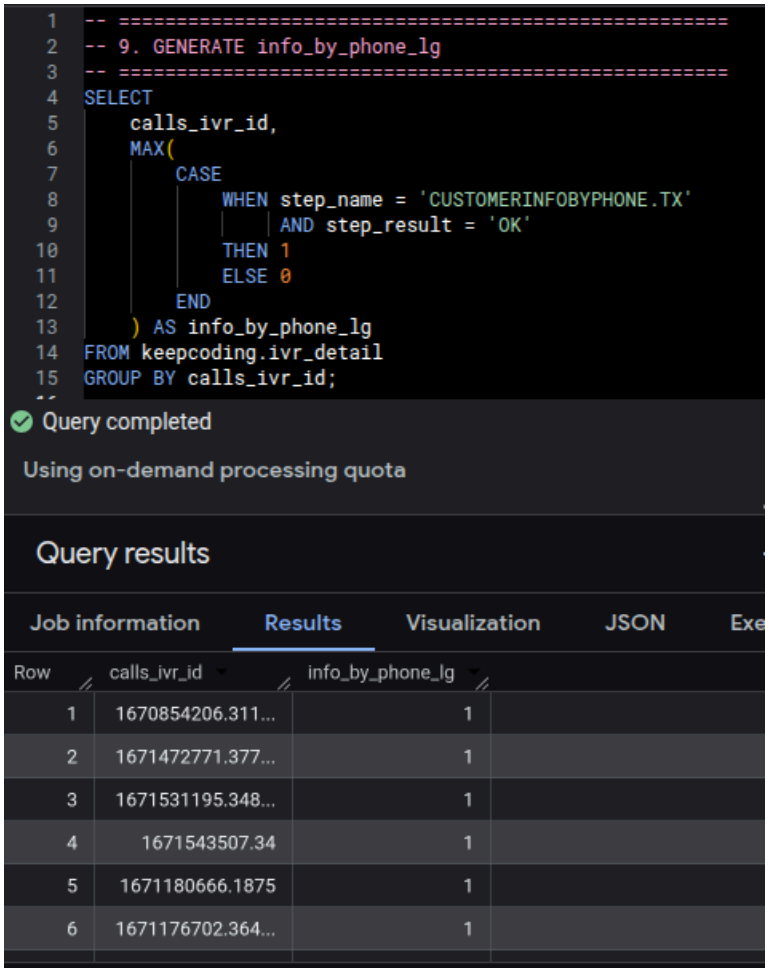



```
-- ==========
-- 8. GENERATE masiva_lg
-- ==========
SELECT
    calls_ivr_id,
    MAX(
        CASE
            WHEN module_name = 'AVERIA_MASIVA' THEN 1
            ELSE 0
        END
    ) AS masiva_lg
FROM keepcoding.ivr_detail
GROUP BY calls_ivr_id;
```

Challenge 9: Generate info_by_phone_lg

Here I am creating a flag to indicate if the customer was successfully identified by their phone number during the call.

I am grouping the records by ivr_id and looking for the step CUSTOMERINFOBYPHONE.TX with result OK. I am using MAX(IF(...)) to ensure that the flag is 1 if this occurs in any step of the call.

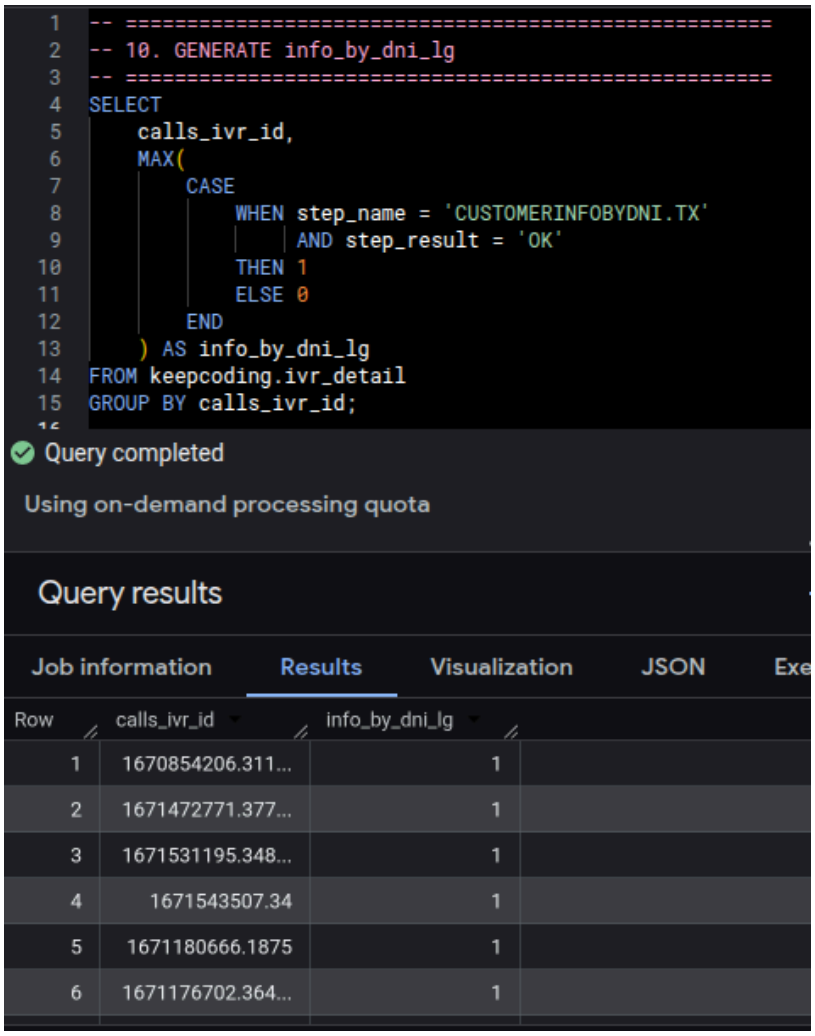



```
-- ==========
-- 9. GENERATE info_by_phone_lg
-- ==========
SELECT
    calls_ivr_id,
    MAX(
        CASE
            WHEN step_name = 'CUSTOMERINFOBYPHONE.TX'
                AND step_result = 'OK'
            THEN 1
            ELSE 0
        END
    ) AS info_by_phone_lg
FROM keepcoding.ivr_detail
GROUP BY calls_ivr_id;
```

Challenge 10: Generate info_by_dni_lg

To create a flag indicating whether the customer was successfully identified using their DNI, search for the step CUSTOMERINFOBYDNI.TX with result OK for each call.

Again, I use MAX(IF(...)) so that the flag becomes 1 if the condition occurs in any step associated with that ivr_id.




```
-- ==========
-- 10. GENERATE info_by_dni_lg
-- ==========
SELECT
    calls_ivr_id,
    MAX(
        CASE
            WHEN step_name = 'CUSTOMERINFOBYDNI.TX'
                AND step_result = 'OK'
            THEN 1
            ELSE 0
        END
    ) AS info_by_dni_lg
FROM keepcoding.ivr_detail
GROUP BY calls_ivr_id;
```

Challenge 11: Generate repeated_phone_24H & cause_recall_phone_24H

The goal here is to flag cases whether the same phone has another call within 24 hours before or after a given call.

I am JOINing the table ivr_detail with itself, using two aliases (‘calls’ and ‘recalls’) so that rows with the same phone number but different ivr_id can be compared.

DATE_DIFF measures the time difference in seconds between the two calls. Two flags are created: one for calls within the previous 24 hours and another for calls within the following 24 hours.

```
-- ============================================================
-- 11. GENERATE repeated_phone_24H AND cause_recall_phone_24H
-- ============================================================
SELECT calls.calls_ivr_id,
       MAX(IF(DATE_DIFF(calls.calls_start_date, recalls.calls_start_date, SECOND) BETWEEN 1 AND 24*60*60, 1, 0)) AS repeated_phone_24H,
       MAX(IF(DATE_DIFF(calls.calls_start_date, recalls.calls_start_date, SECOND) BETWEEN -24*60*60 AND -1, 1, 0)) AS cause_recall_phone_24H
FROM keepcoding.ivr_detail calls
LEFT JOIN keepcoding.ivr_detail recalls
       ON calls.calls_phone_number <> 'UNKNOWN'
      AND calls.calls_phone_number = recalls.calls_phone_number
      AND calls.calls_ivr_id <> recalls.calls_ivr_id
GROUP BY calls.calls_ivr_id;
```

Query completed

Using on-demand processing quota

Query results

+ Create conversation | Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	calls_ivr_id	repeated_phone_...	cause_recall_pho...
1	1671176702.364...	0	1
2	1671044264.317...	0	1
3	1671178821.364...	0	1
4	1671457644.345...	0	0
5	1671448914.2854	0	0
6	1670926642.345...	0	1
7	1670844672.310...	0	1

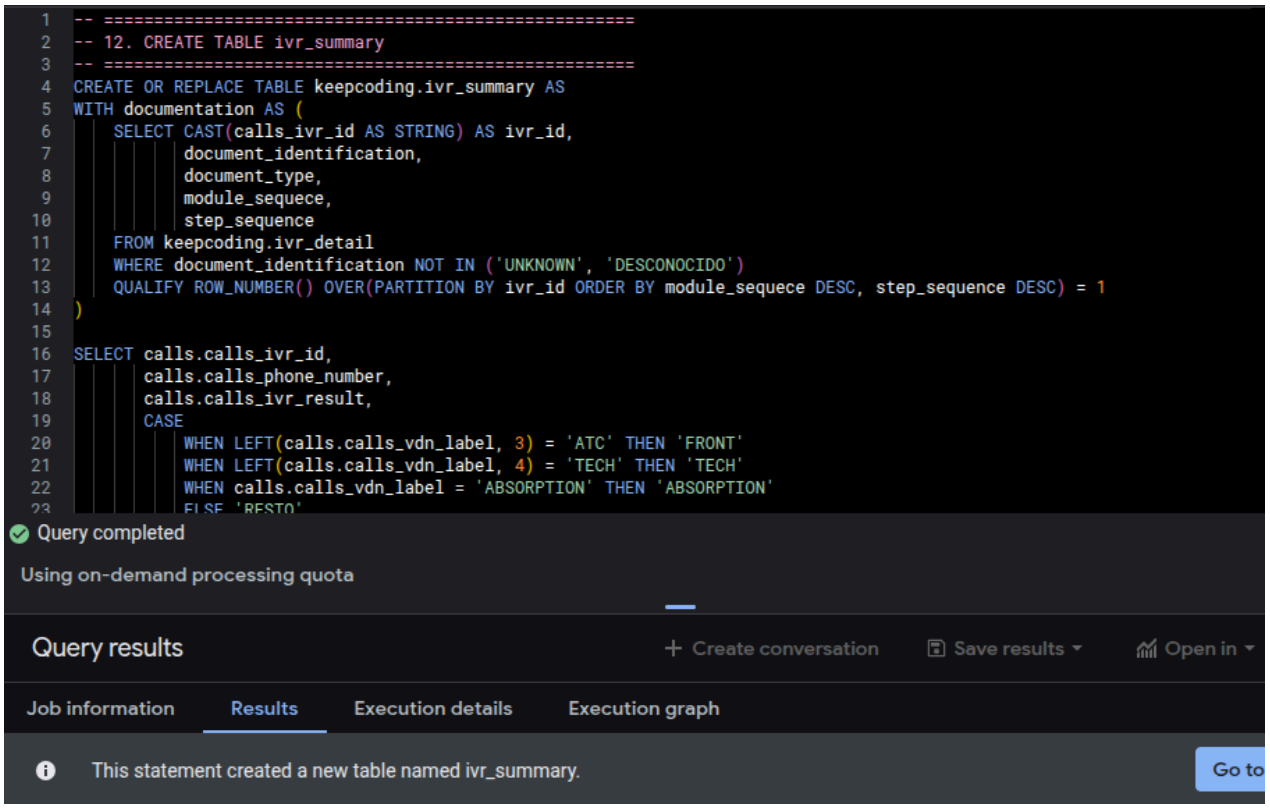

```
-- ============================================================
-- 11. GENERATE repeated_phone_24H AND cause_recall_phone_24H
-- ============================================================
SELECT calls.calls_ivr_id,
       MAX(IF(DATE_DIFF(calls.calls_start_date, recalls.calls_start_date, SECOND) BETWEEN 1 AND 24*60*60, 1, 0)) AS repeated_phone_24H,
       MAX(IF(DATE_DIFF(calls.calls_start_date, recalls.calls_start_date, SECOND) BETWEEN -24*60*60 AND -1, 1, 0)) AS cause_recall_phone_24H
FROM keepcoding.ivr_detail calls
LEFT JOIN keepcoding.ivr_detail recalls
       ON calls.calls_phone_number <> 'UNKNOWN'
      AND calls.calls_phone_number = recalls.calls_phone_number
      AND calls.calls_ivr_id <> recalls.calls_ivr_id
GROUP BY calls.calls_ivr_id;
```

Challenge 12: Create table ivr_summary

The goal is to create a table with one row per call that shows the most important information.

The query combines data from ivr_detail, calculates flags like masiva_lg, info_by_phone_lg, and info_by_dni_lg, and uses a temporary datatable (a WITH CTE) to pick the most appropriate document identification and type.

I am using aggregation functions like MAX(), IFNULL(), together with conditional logical statements, to make sure that for each call we get one clear, complete record with all the correct values.



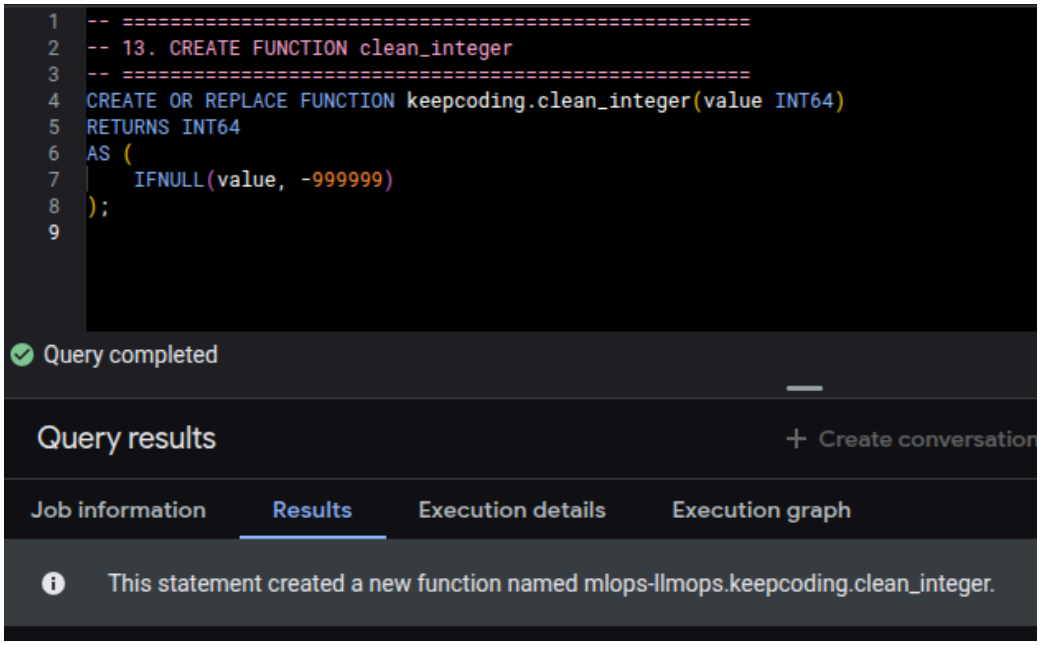

```
-- ==========
-- 12. CREATE TABLE ivr_summary
-- ==========
CREATE OR REPLACE TABLE keepcoding.ivr_summary AS
WITH documentation AS (
   SELECT CAST(calls_ivr_id AS STRING) AS ivr_id,
         document_identification,
         document_type,
         module_sequece,
         step_sequence
   FROM keepcoding.ivr_detail
   WHERE document_identification NOT IN ('UNKNOWN', 'DESCONOCIDO')
   QUALIFY ROW_NUMBER() OVER(PARTITION BY ivr_id ORDER BY module_sequece DESC,
step_sequence DESC) = 1
)

SELECT calls.calls_ivr_id,
      calls.calls_phone_number,
      calls.calls_ivr_result,
```

```
    CASE
      WHEN LEFT(calls.calls_vdn_label, 3) = 'ATC' THEN 'FRONT'
      WHEN LEFT(calls.calls_vdn_label, 4) = 'TECH' THEN 'TECH'
      WHEN calls.calls_vdn_label = 'ABSORPTION' THEN 'ABSORPTION'
      ELSE 'RESTO'
    END AS vdn_aggregation,
    calls.calls_start_date,
    calls.calls_end_date,
    calls.calls_total_duration,
    calls.calls_customer_segment,
    calls.calls_ivr_language,
    calls.calls_steps_module,
    calls.calls_module_aggregation,
    IFNULL(documentation.document_type, 'DESCONOCIDO') AS document_type,
    IFNULL(documentation.document_identification, 'DESCONOCIDO') AS document_identification,
    IFNULL(MAX(NULLIF(calls.customer_phone, 'UNKNOWN')), 'DESCONOCIDO') AS customer_phone,
    IFNULL(MAX(NULLIF(calls.billing_account_id, 'UNKNOWN')), 'DESCONOCIDO') AS billing_account_id,
    MAX(IF(calls.module_name = "AVERIA_MASIVA", 1, 0)) AS masiva_lg,
    MAX(IF(calls.step_name = 'CUSTOMERINFOBYPHONE.TX' AND calls.step_result = 'OK', 1, 0)) AS
info_by_phone_lg,
    MAX(IF(calls.step_name = 'CUSTOMERINFOBYDNI.TX' AND calls.step_result = 'OK', 1, 0)) AS
info_by_dni_lg,
    MAX(IF(DATE_DIFF(calls.calls_start_date, recalls.calls_start_date, SECOND) BETWEEN 1 AND
24*60*60, 1, 0)) AS repeated_phone_24H,
    MAX(IF(DATE_DIFF(calls.calls_start_date, recalls.calls_start_date, SECOND) BETWEEN -24*60*60
AND -1, 1, 0)) AS cause_recall_phone_24H
FROM keepcoding.ivr_detail calls
LEFT JOIN documentation
    ON CAST(calls.calls_ivr_id AS STRING) = documentation.ivr_id
LEFT JOIN keepcoding.ivr_detail recalls
    ON calls.calls_phone_number <> 'UNKNOWN'
   AND calls.calls_phone_number = recalls.calls_phone_number
   AND calls.calls_ivr_id <> recalls.calls_ivr_id
GROUP BY calls.calls_ivr_id,
      calls.calls_phone_number,
      calls.calls_ivr_result,
      vdn_aggregation,
      calls.calls_start_date,
      calls.calls_end_date,
      calls.calls_total_duration,
      calls.calls_customer_segment,
      calls.calls_ivr_language,
      calls.calls_steps_module,
      calls.calls_module_aggregation,
      document_type,
      document_identification;
```

Challenge 13: Create function clean_integer

In this last challenge, I am creating a reusable function called clean_integer inside the ‘keepcoding’ dataset. The function takes one number (‘value’) as input and checks if it is NULL. If so, it returns -999999; otherwise it returns the original number.

Basically, it is a function that “cleans” all missing values, replacing them by the dummy -999999.




```
-- ==========
-- 13. CREATE FUNCTION clean_integer
-- ==========
CREATE OR REPLACE FUNCTION keepcoding.clean_integer(value INT64)
RETURNS INT64
AS (
   IFNULL(value, -999999)
);
```